
Does LLM-Generated Synthetic Data Help or Hurt Classical ML Training?

Sriram Kadiyala¹ Emon Steadman¹ Nisarga Bhaskar¹ Vikash Churiwala¹

Abstract

The “model collapse” phenomenon, where models trained on synthetic data degrade over generations, is well-studied for large language models. But what happens when LLM-generated synthetic data is used to train *classical* ML models? We systematically investigate whether synthetic tabular data generated by Claude Opus 4.6 can substitute for, augment, or hurt the performance of logistic regression, SVM, XGBoost, and MLP on three UCI datasets of varying size. We compare two LLM prompting strategies (distribution-constrained and sample-based) against CTGAN as a traditional baseline, across five real-to-synthetic mixing ratios in both full-data and low-data regimes. Our key finding is that LLM synthetic data significantly outperforms real data on small datasets in low-data settings (0.879 vs. 0.787 accuracy on Heart Disease with logistic regression), while CTGAN collapses to near-random performance. On larger datasets, synthetic data acts as a neutral substitute with slight degradation. We also find that prompting strategy matters: distribution constraints improve marginal fidelity, while sample-based prompts improve inter-column logical consistency, revealing a tradeoff that practitioners must navigate.

1. Introduction

As large language models become increasingly capable at generating structured text, a natural question arises: can LLMs generate useful *tabular* training data for classical machine learning pipelines? This question has practical implications, as tabular data remains the dominant format in industry applications, and classical models like XGBoost and logistic regression still outperform deep learning on typical tabular tasks (Grinsztajn et al., 2022).

The “model collapse” phenomenon (Borisov et al., 2023) describes how models trained on synthetic data can degrade over successive generations. While this is well-studied for LLMs training on LLM outputs, the effect on classical ML models trained on LLM-generated tabular data remains underexplored.

Recent work has begun to address LLM-based tabular data generation. GReaT (Borisov et al., 2023) showed that fine-tuned GPT-2 can generate realistic tabular rows by serializing them as text. CLLM (Seedat et al., 2024) demonstrated that LLMs can augment ultra-low-data regimes when combined with data curation. Xu et al. (Xu et al., 2024) provided a critical perspective, finding that LLMs struggle with “functional dependencies” between columns. However, these studies primarily evaluate synthetic data quality through neural model performance or statistical similarity metrics.

Our work differs in three ways. First, we evaluate downstream performance on *classical* ML models (logistic regression, SVM, XGBoost, MLP), which remain the standard for tabular tasks. Second, we use an off-the-shelf LLM (Claude Opus 4.6) with zero fine-tuning, reflecting the practical scenario where a practitioner simply prompts an LLM. Third, we systematically compare two prompting strategies as a controlled experimental variable: distribution-constrained prompts (explicit frequency targets) versus sample-based prompts (100 real rows as in-context examples, no constraints). We benchmark both against CTGAN (Xu et al., 2019), a widely-used GAN-based tabular data generator.

2. Methods

2.1. Datasets

We use three UCI datasets (Dua & Graff, 2017) with varying characteristics:

Adult Income (48,842 rows, 14 features): Binary classification of income above or below \$50K. Mix of categorical and numerical features. Large enough that real data should be sufficient.

Credit Card Default (30,000 rows, 23 features): Binary classification of credit card default. Contains six months of temporal payment history with inter-column dependencies (e.g., payment amounts relative to bill amounts).

Heart Disease (303 rows, 13 features): Binary classification of heart disease presence from the Cleveland subset. Small enough that data augmentation should meaningfully impact performance.

2.2. Synthetic Data Generation

We generated synthetic data using three approaches:

LLM-Constrained: We provided Claude Opus 4.6 with the dataset schema, allowed values, and explicit distribution targets (e.g., “approximately 46% of rows should have hours-per-week = 40”) derived from the real data. The LLM was instructed to generate CSV rows matching these constraints.

LLM-Sample: We provided the same schema plus 100 real sample rows as in-context examples, but *no* distribution constraints. The LLM was asked to generate new rows following the patterns in the examples.

CTGAN: We used the Conditional Tabular GAN (Xu et al., 2019) trained on the real training data as a traditional baseline.

Synthetic rows were generated in batches of 500, with 5,000 rows generated per dataset (except Heart Disease, which yielded $\sim 4,200$ unique rows due to duplicate collisions). After generation, an automated validation pipeline checked each batch for exact memorization against the real data, numerical range violations, categorical value validity, inter-column logical consistency (e.g., $\text{PAY_AMT} \leq \text{BILL_AMT}$), education-number mapping accuracy, and distribution alignment. Batches failing memorization checks were discarded; Heart Disease required discarding three batches with 17–84% exact matches.

2.3. Experimental Design

We designed two experimental regimes:

Full Data Regime: Models trained on 5,000 data points at varying ratios of real to synthetic data (100/0, 75/25, 50/50, 25/75, 0/100). For Heart Disease, the training size was capped at $5\times$ the real training set (1,210 rows).

Low Data Regime: Real data was restricted to $\min(10\%$ of full size, 500) rows, simulating scarce-data scenarios. Synthetic augmentation was allowed to double this amount, keeping total training size at $2\times$ the real data cap. For Heart Disease, this yields 30 real rows with a target of 60.

Table 1. Experimental conditions for full and low data regimes.

Ratio	Full Data			Low Data		
	Real	Syn	Total	Real	Syn	Total
100/0	5000	–	5000	500	–	500
75/25	3750	1250	5000	500	250	750
50/50	2500	2500	5000	500	500	1000
25/75	1250	3750	5000	250	750	1000
0/100	–	5000	5000	–	1000	1000

All models were evaluated on a held-out real test set. For Adult, we used the UCI-provided test partition; for Credit

and Heart, we used a 20% random split. Each experiment was repeated with 5 random seeds, and we report mean $\pm$ standard deviation. Data processing included StandardScaler on all features and one-hot encoding of categorical features using encoders fit on the real training data.

3. Results

In all figures, a ratio of 1.0 on the x-axis represents purely real data, decreasing leftward as synthetic data replaces real data.

3.1. Logistic Regression

We used scikit-learn’s LogisticRegression with lbfgs solver and max_iter=1000.

Across all three datasets in the full data regime, performance was stable. Adult accuracy remained within 82–85% regardless of synthetic source or ratio. Credit showed similar stability at 79–81%.

The most notable results came from Heart Disease. In the low data regime (30 real rows, target size 60), pure LLM-constrained synthetic data achieved 0.879 accuracy and 0.958 AUC, significantly outperforming the pure real baseline of 0.787 accuracy and 0.877 AUC. LLM-sample performed similarly at 0.856 accuracy and 0.957 AUC. CTGAN collapsed to near-random performance (0.423 accuracy, 0.394 AUC).

In the full regime for Heart, a 50/50 LLM-constrained mix achieved the best accuracy at 0.915 with 0.973 AUC, exceeding the pure real baseline (0.836 accuracy, 0.939 AUC). LLM-constrained outperformed LLM-sample on Heart, while on Adult and Credit, LLM-sample slightly edged out LLM-constrained at pure synthetic ratios. We hypothesize this is because logistic regression learns linear decision boundaries sensitive to marginal distributions, where constrained prompts excel.

CTGAN was competitive with or outperformed LLM methods on larger datasets but consistently underperformed on Heart, indicating GANs require sufficient training data to learn distributions, while LLMs bring pre-trained prior knowledge that compensates for data scarcity.

3.2. XGBoost

XGBoost (Chen & Guestrin, 2016) uses second-order gradient boosting with Newton-style approximations to form a highly efficient tree boosting algorithm. For selecting tree hyperparameters, 5-fold cross-validation was performed on number of trees, max tree depth, learning rate, subsample size, and per-tree features used. The hyperparameters were chosen based on the real data, then the other real/synthetic mixtures used these same parameters (on a per-dataset ba-

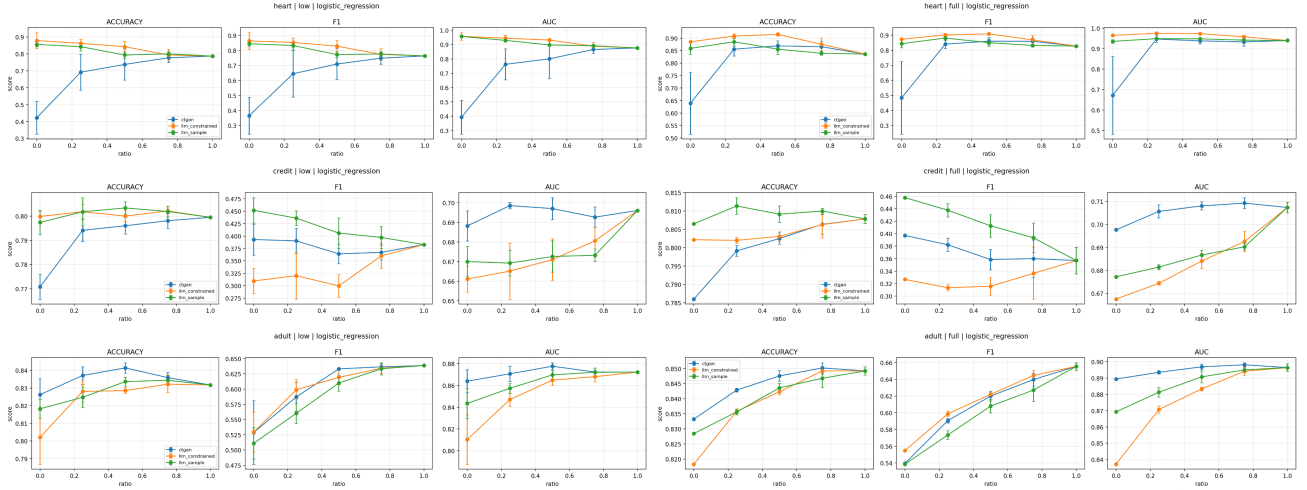


Figure 1. Logistic regression results across all six dataset-regime combinations. Top row: Heart Disease (low and full). Middle row: Credit Card Default. Bottom row: Adult Income. LLM methods dramatically outperform CTGAN on Heart low regime while all methods converge on larger datasets.

sis).

When training in the full data regime, introducing synthetic data slightly decreased performance or had no measurable impact. At a 25:75 real/synthetic split, accuracy degradation was typically small. For example, on the Adult dataset accuracy fell from 0.858 to 0.831. On the Credit dataset the drop was negligible, 0.814 to 0.804. CTGAN performed comparably to LLM-generated data at moderate synthetic ratios, but when training on purely synthetic data CTGAN performance degraded sharply. On the Heart dataset, pure CTGAN data resulted in an accuracy of 0.479 versus 0.846 for `llm_constrained` and 0.859 for `llm_sample`. Between the two LLM strategies, `llm_constrained` slightly outperformed `llm_sample` on Adult across higher real-data ratios (0.855 vs 0.851 accuracy at 75:25), while on Credit the two strategies performed nearly identically, with no consistent winner across ratios.

When training models in the low data regime, synthetic data augmentation provided a modest but consistent benefit, where a 50:50 real/synthetic mixture yielded peak performance. On the Heart dataset, the pure real baseline gave 0.816 accuracy, while the best 50:50 result (using `llm_constrained` data) reached 0.866 accuracy. On the Credit dataset, the benefit was smaller and less consistent; accuracy ranged from 0.800 to 0.807 across all ratios and methods, with CTGAN reaching 0.807 at a 50:50 mix versus a 0.800 baseline. By contrast, CTGAN compared poorly when trained on purely synthetic Heart data, dropping to 0.495 accuracy against that same 0.816 real baseline.

3.3. Support Vector Machine

We used scikit-learn’s `LinearSVC` with `max_iter=10000`, applying `StandardScaler` and consistent one-hot encoding across real and synthetic datasets. In the full data regime, SVM performance was generally stable across all datasets, though slightly more sensitive to synthetic data than logistic regression. On Adult, accuracy remained around 83.5%, with small but consistent declines as the proportion of synthetic data increased. On Credit, performance was similarly stable (80.75% accuracy, AUC 0.70), with minimal differences between `llm_constrained`, `llm_sample`, and CTGAN at moderate ratios. Overall, synthetic data had limited impact in large-data settings, acting mostly as a neutral substitute with slight degradation at high synthetic proportions.

The most notable effects appeared on the Heart Disease dataset and in the low data regime. With limited real data, synthetic augmentation often improved both accuracy and AUC, particularly with LLM-generated data at intermediate ratios (e.g., 50/50), which sometimes exceeded the pure real baseline. `llm_constrained` generally performed best, suggesting that matching marginal distributions is important for SVM’s margin-based learning. In contrast, CTGAN was less reliable, especially when used as the sole data source, occasionally leading to near-random performance. Overall, these results suggest that SVM benefits from high-quality synthetic data in low-data settings, but gains depend heavily on how well the synthetic samples preserve class structure and decision boundaries.

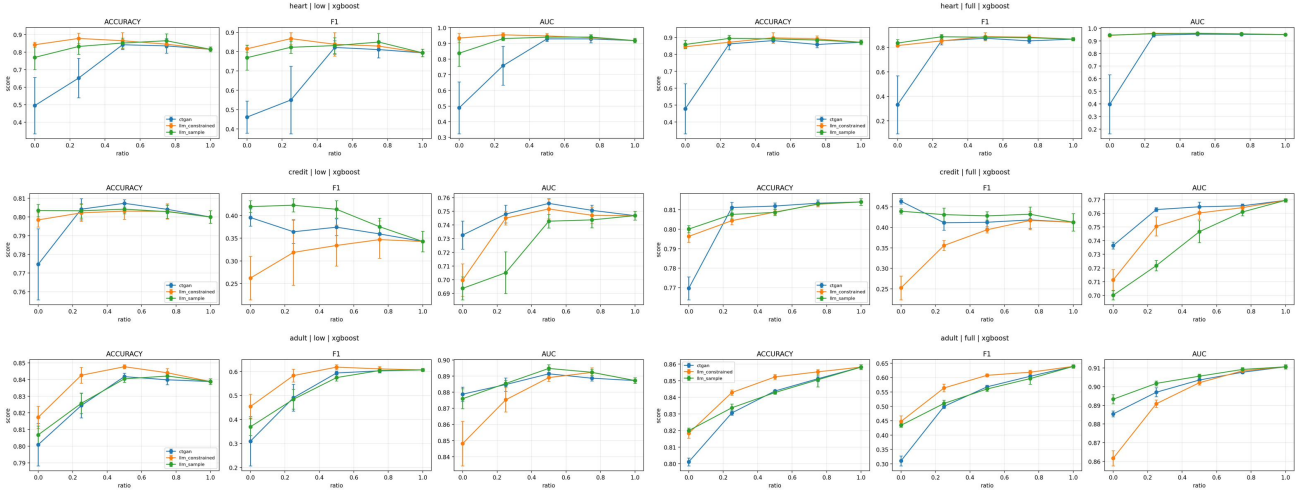


Figure 2. XGBoost results across all six dataset-regime combinations. Top row: Heart Disease (low and full). Middle row: Credit Card Default. Bottom row: Adult Income.

3.4. Neural Network

We trained a multi-layer perceptron (MLP) with two hidden layers (128 and 64 units, ReLU activation, Adam optimizer, early stopping) using the same experimental protocol as the other models. On the Adult and Credit datasets under the full-data regime, MLP performance was stable across all synthetic data sources, with accuracy within 80-85% for Adult and 79-81% for Credit. The most striking results came from Heart Disease. In the full regime, pure real data yielded only 0.744 mean accuracy and 0.817 AUC, the lowest baseline among all four models. LLM-constrained synthetic data dramatically improved performance at every mixing ratio, peaking at ratio=0.25 with 0.928 accuracy. Even purely synthetic LLM-constrained data (ratio=0.0) achieved 0.908 accuracy and 0.974 AUC, far exceeding the real baseline. CTGAN collapsed to 0.590 accuracy at ratio=0.0. In the low data regime for Heart, the real-only baseline achieved 0.761 accuracy. LLM-constrained augmentation at ratio=0.5 improved this to 0.807, while purely synthetic LLM-constrained reached 0.761. CTGAN was unreliable, with high variance across seeds. On Credit low regime, llm_constrained showed a notable failure mode: F1 dropped to near-zero at ratio=0.0 (mean F1 = 0.002), indicating the model predicted only the majority class. LLM-sample did not exhibit this issue, maintaining F1 around 0.403 at the same ratio. These results suggest MLP benefits substantially from LLM synthetic data on small datasets, with even larger gains than logistic regression or SVM. However, MLP is also the most volatile, with high seed-to-seed variance, particularly on Heart where individual seeds ranged from 0.459 to 0.902 accuracy within the same condition.

3.5. Cross-Model Summary

Across all models, a consistent pattern emerges: LLM synthetic data helps in low-data regimes (especially Heart Disease) but provides diminishing or negative returns when real data is abundant. CTGAN is competitive on larger datasets but collapses on small ones. XGBoost is the most sensitive to synthetic data quality, likely because tree-based splits depend on precise inter-column relationships that LLMs fail to capture. Logistic regression and SVM are more forgiving, as their decision boundaries depend primarily on individual feature weights or margins rather than complex feature interactions.

The prompting strategy comparison reveals that llm_constrained generally performs best on the smallest dataset (Heart), while llm_sample shows competitive or slightly better performance on larger datasets where inter-column consistency matters more than marginal accuracy.

4. Process

4.1. Data Generation Challenges

Distribution constraints are essential. Without them, the first Adult Income batch had completely wrong distributions (hours=40 at 2.7% instead of 46.7%). Adding explicit frequency targets in the prompt fixed this almost entirely.

Schema verification against raw data is critical. Heart Disease had three columns (cp, slope, thal) with different value sets than what online descriptions indicated. Kaggle versions re-encode columns; the raw UCI file is the ground truth.

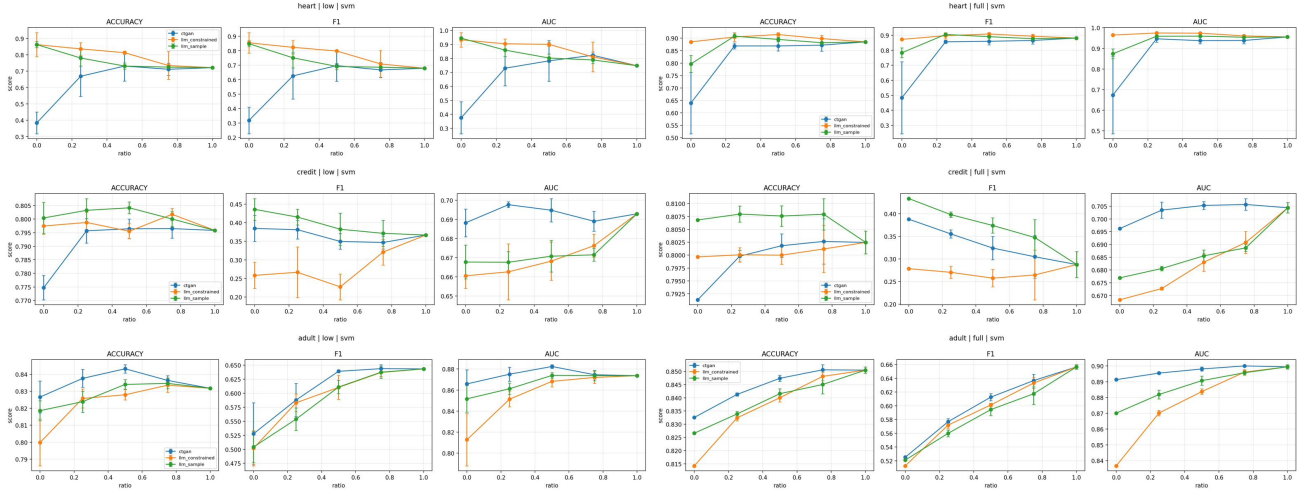


Figure 3. SVM results across all six dataset-regime combinations. Top row: Heart Disease (low and full). Middle row: Credit Card Default. Bottom row: Adult Income. CTGAN shows high variance and collapse on Heart low regime.

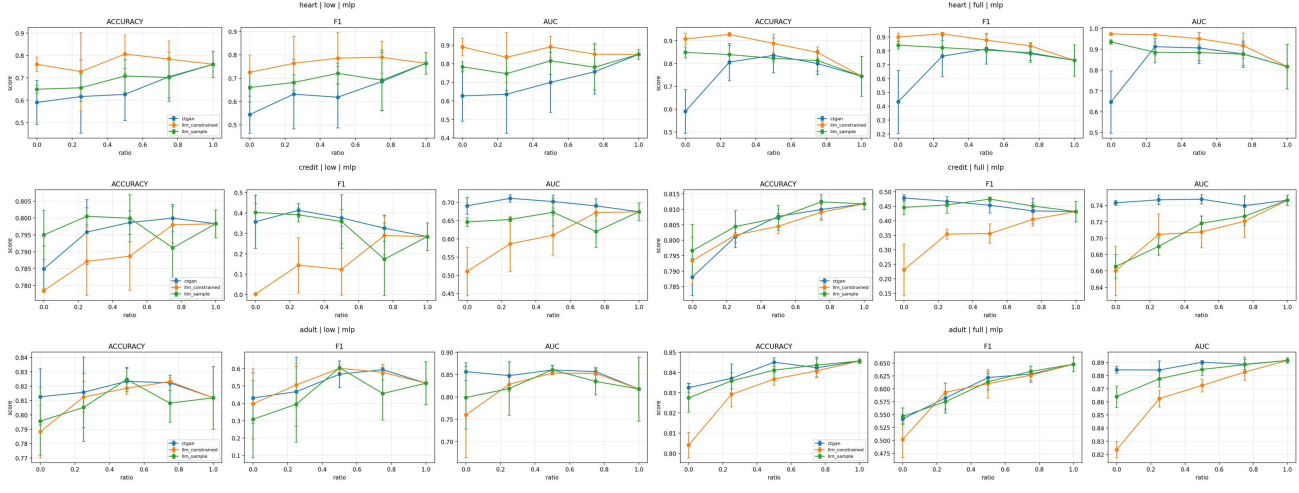


Figure 4. MLP results across all six dataset-regime combinations. Top row: Heart Disease (low and full). Middle row: Credit Card Default. Bottom row: Adult Income. LLM-constrained dominates on Heart while MLP shows highest variance among all models.

Batch-to-batch variation is real but averages out. Individual batches varied (e.g., hours=40 ranged from 44% to 65%), but the combined 5,000-row dataset was close to targets.

Logical consistency is the weak point. LLMs can match marginal distributions when instructed, but maintaining inter-column relationships (like $PAY_AMT \leq BILL_AMT$) proved difficult. On the Credit dataset, 3,029 out of 30,000 payment-bill column pairs violated this constraint in the LLM-constrained data.

Memorization risk for small, well-known datasets. Heart Disease required discarding multiple batches due to 17–

84% exact matches with real data. Fresh chat sessions and explicit “do not reproduce” instructions helped but did not eliminate the problem.

Prompt strategy tradeoff. LLM-constrained prompts produced better marginal distributions; sample-based prompts produced better inter-column logic (PAY>BILL violations dropped from 3,029 to 2,458 on Credit). No single approach optimizes both.

LLM output limitations. Claude Opus 4.6 maxed out at approximately 500 rows per response regardless of the requested amount. On Heart Disease, the LLM repeatedly attempted to write Python scripts to generate data program-

matically rather than outputting CSV rows directly, requiring explicit prompt instructions to prevent this behavior. Validation scripts should be written before generation starts to catch issues immediately.

4.2. Low Data Regime Design

An initial implementation set the low-data fraction at 10% of the real training data without a cap, giving 3,256 rows for Adult, which is not meaningfully “low.” This was corrected to $\min(10\%, 500)$ rows with a consistent $2\times$ augmentation multiplier across all datasets. This correction was identified during collaborative debugging and applied before final results were generated.

5. Contributions

Sriram Kadiyala: Designed and implemented the LLM synthetic data generation pipeline for all three datasets using both prompting strategies. Wrote validation scripts for memorization detection, range checks, categorical validity, and distribution alignment. Built the experimental pipeline (`logistic_regression.py`) that served as the template for all four models. Ran all 450 logistic regression experiments. Conducted literature review. Drafted logistic regression results section and data generation methodology.

Emon Steadman: Built the data preprocessing pipeline (`preprocess.py`) including train/test splitting, one-hot encoding, and dataset mixing utilities. Generated CTGAN synthetic data for all three datasets. Ran all XGBoost experiments with cross-validated hyperparameter tuning. Drafted XGBoost results section and experimental design methodology. Identified and proposed the 500-row low-data cap. Conducted literature review.

FNU Nisarga Bhaskar: Ran all SVM experiments. Built the analysis and plotting script (`analyze_results.py`) and generated all result visualizations for the report. Created check-in presentation slides and led the first check-in presentation. Drafted SVM results section. Conducted literature review.

Vikash Churiwala: Ran all MLP/neural network experiments. Drafted neural network results section.

We used scikit-learn, XGBoost, imbalanced-learn, and the CTGAN (SDV) library. Datasets were sourced from the UCI Machine Learning Repository

6. AI Usage

Claude (Anthropic) was used extensively throughout this project:

Data Generation: Claude Opus 4.6 was the primary LLM used for synthetic tabular data generation. Two prompting strategies (distribution-constrained and sample-based) were

tested as experimental variables.

Coding Assistance: Claude assisted with writing the experimental pipeline, debugging environment issues (numpy/scipy/pandas version conflicts), and fixing data preprocessing bugs (header parsing, NaN handling for CTGAN).

All results reported in this paper were generated by our own code and verified against raw experimental outputs. We take full responsibility for the accuracy of all reported numbers.

7. Conclusions

Our results demonstrate that LLM-generated synthetic tabular data can be a viable augmentation strategy for classical ML in low-data regimes, particularly when real training data is scarce. On the Heart Disease dataset (303 rows), pure LLM-generated synthetic data achieved higher mean performance than real-data baselines across all four models, with logistic regression achieving 0.879 vs. 0.787 accuracy. One possible explanation is that the LLM’s pretraining exposed it to medically relevant statistical patterns.

However, when real data is abundant, synthetic data provides diminishing returns and can hurt performance, especially for tree-based models like XGBoost that are sensitive to inter-column dependencies. CTGAN remains competitive on larger datasets but fails catastrophically on small ones.

The choice of prompting strategy matters: distribution constraints improve marginal fidelity, while sample-based prompts improve logical consistency. Practitioners should select their strategy based on what their downstream model requires. This effect was most pronounced on Credit Card Default, where sample-based prompts reduced inter-column violations (e.g., payment exceeding bill amount) and yielded higher F1 scores, while constrained prompts performed better on Heart Disease where marginal distributions mattered more.

Limitations and Future Work. We tested only one LLM (Claude); extending to GPT-4, Gemini, and open-source models would strengthen generalizability. Our low-data regime uses a fixed real-data subsample across seeds, underestimating variance. Future work could explore bias in LLM-generated data, test with more prompting strategies (e.g., chain-of-thought, varying sample sizes), and evaluate on a broader range of datasets and model families.

Code and Data: All code, synthetic datasets, and experimental results are available at: <https://github.com/sriramkadiyala6/synthetic-data-classical-ml>

References

- Borisov, V., Seßler, K., Leemann, T., Pawelczyk, M., and Kasneci, G. Language models are realistic tabular data generators. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.06280>.
- Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. doi: 10.1145/2939672.2939785.
- Dua, D. and Graff, C. UCI machine learning repository, 2017. URL <http://archive.ics.uci.edu/ml>.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2207.08815>.
- Seedat, N., Huynh, N., van Breugel, B., and van der Schaar, M. Curated LLM: Synergy of LLMs and data curation for tabular augmentation in ultra low-data regimes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2312.12112>.
- Xu, L., Skoularidou, M., Cuesta-Infantes, A., and Veeramachaneni, K. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL <https://arxiv.org/abs/1907.00503>.
- Xu, S., Lee, C.-T., Sharma, M., Yousuf, R. B., Muralidhar, N., and Ramakrishnan, N. Why LLMs are bad at synthetic table generation (and what to do about it). *arXiv preprint arXiv:2406.14541*, 2024. URL <https://arxiv.org/abs/2406.14541>.